Expose whether atomic notifying operations are lock free

Document #:	P3255R1
Date:	2024-07-16
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Brian Bi <bbi10@bloomberg.net>

Contents

§ 1 Revision history

§ 1.1 R1

— Added `wait_is_signal_safe`, `wait_is_always_signal_safe`, and `std::atomic_is_signal_safe`.

— Miscellaneous wording fixes.

— Now ready for LEWG review after being forwarded from SG1.

§ 2 Abstract

The atomic notifying functions for `std::atomic_flag` are not always lock-free even though `std::atomic_flag` is specified to always be lock-free. Therefore, use of `std::atomic_flag::notify_one` or `std::atomic_flag::notify_all` may cause unpredictable behavior when called in a signal handler, even though the Standard currently claims that these functions are signal-safe. It would be useful for signal handlers to know for sure whether they can safely notify a `std::atomic_flag`. Therefore, I propose the introduction of member constant `notify_is_always_lock_free`, member function `notify_is_lock_free`, and free function `std::atomic_notify_is_lock_free` for `std::atomic_flag`. I also propose the same for `std::atomic` and `std::atomic_ref` specializations in case the lock-free property of their notifying functions differ from those of their other functions such as loads and stores. Similarly, in the case of the atomic waiting operations, it would be useful to know whether they are safe to call in signal handlers (despite not being lock-free).

§ 3 The problem

Atomic notifying operations for `std::atomic_flag` were originally proposed by [P0514R0]. In that paper, the authors provided a reference implementation and wrote that their current implementation was not lock-free. In the following revision, [P0514R1], the authors revised their proposal to no longer propose the addition of the waiting/notifying interface to `std::atomic_flag`, writing that "lock-freedom is guaranteed to `atomic_flag` and could not be preserved with the extension". That version of the paper instead proposed a waiting/notifying interface only for semaphore types. However, the atomic waiting/notifying interface was eventually added to `std::atomic_flag` and `std::atomic` by [P1135R6], and to `std::atomic_ref` by [P1643R1].

It was pointed out to me that the waiting functions are clearly not lock-free because they may block the calling thread. Whether or not the notifying functions are lock-free is a much more interesting question. The notifying functions can always be made lock-free by implementing them as no-ops while the waiting threads spin, waiting for the value to change[1], which is one of the approaches used by the reference implemention provided with P0514R0 [refimpl]. On Linux and Windows, which provide operating system support for notifying and waiting on objects up to a certain size, the reference implementations of the notifying functions consist of a single system call for each function, which makes them lock-free.

In practice, however, implementations of the Standard Library fall back to the "array of condvars" strategy at least some of the time, rather than spinning:

— libstdc++ uses an array of condvars on platforms other than Linux, as well as on Linux for atomic objects that are too large for the futex API.

— libc++ currently always uses an array of condvars; the use of futexes or other operating system specific APIs does not appear to have been implemented yet.

— Although the Microsoft STL supports only Windows, it supports versions of Windows that do not provide system calls for waiting and notifying, so must fall back to an array of condvars on such platforms, even for `std::atomic_flag`.

Therefore, in practice, the `notify_one` and `notify_all` functions for `std::atomic_flag` are not always lock-free even though the Standard specifies that all operations on `std::atomic_flag` shall be lock-free (§33.5.10 [atomics.flag][2]p2). I think that the implementations correctly implement the intent of the Standard, but the wording of the Standard erroneously requires implementations lacking operating system support for waiting and notifying to spin rather than using an array of condvars, considering that notifying a condvar is not, in general, a lock-free operation.

Similarly, for `std::atomic` and `std::atomic_ref`, I believe that the intent of the Standard is that `is_lock_free`, `is_always_lock_free`, and `atomic_is_lock_free` should report whether all operations *except* the waiting and notifying operations are lock-free.

§ 4 Importance of lock-free notifying operations

Fixing the wording to match the intent could be accomplished through an LWG issue: we would simply say that all operations on `std::atomic_flag` other than the waiting and notifying operations are lock-free, and that `is_lock_free`, `is_always_lock_free`, and `atomic_is_lock_free` for `std::atomic<T>` and `std::atomic_ref<T>` report whether all operations other than the waiting and notifying operations are lock-free.

This paper proposes something else in addition to the above. I believe that being able to tell whether the notifying operations are lock-free, at the very least for `std::atomic_flag`, would be extremely useful because no equivalent functionality is currently available for use in signal handlers. Because the set of functions specified by the Standard and by POSIX as signal-safe is so limited—for example, even `printf` is not signal-safe—a programmer who wishes to perform any but the simplest operations in a signal handler for an inherently fatal signal such as SIGSEGV or SIGFPE must generally use the signal handler only to store data to a global variable that some other thread reads and acts on. While it would be acceptable for the signal handler itself to spin (considering that the program cannot continue to function normally anyway), there should not be a thread that spends its entire lifetime spinning while waiting for a global variable to change (indicating that a signal handler has asked it to do something), since in most executions, the signal handler will hopefully not be invoked at all. It is desirable for the thread that performs the actual work to be blocked while waiting to be woken up by the signal handler. Unfortunately, `pthread_cond_signal` is not signal-safe so it cannot be used by the signal handler to wake up another thread, and workarounds must be used such as communicating through the filesystem (*i.e.*, using a pipe or socket). Such workarounds are not only obscure but also cumbersome: they are difficult to implement correctly, resulting in a source of bugs.

§17.13.5 [support.signal] currently implies that the notifying operation of `std::atomic_flag` is safe to call within a signal handler; however, since this safety is considered to be a result of such operations being "plain lock-free atomic operations", and such operations are not always lock-free in practice, unpredictable behavior may occur when such operations are called within a signal handler, despite what the Standard says. Instead of merely amending the Standard to exclude the notifying operations of `std::atomic_flag` from being signal-safe, we should give users a way to determine when those operations *are* signal-safe so that they can rely on defined behavior when performing those operations in signal handlers. On implementations that provide the guarantee that atomic notifying operations are lock-free (and therefore signal safe), those operations can become the preferred means for a signal handler to wake up another thread.

§ 5 `notify_is_always_lock_free` versus `notify_is_lock_free`

[P0152R1] discusses the rationale for both the older runtime functions `is_lock_free` and `std::atomic_is_lock_free` and the newer constant `is_always_lock_free`. For the notifying operations, similar considerations apply. If a program is compiled for both old and new versions of an operating system, but only new versions have the necessary support for lock-free notifying operations, then `notify_is_always_lock_free` will not be true during the compilation, and the program will have to perform a runtime check. On the other hand, if the programmer simply doesn't want to support any platforms that don't provide a lock-free atomic notifying operation, they might wish to `static_assert(std::atomic_flag::notify_is_always_lock_free);` this assertion might pass if the user has configured their toolchain to target only newer versions of the target operating system. For this reason, this paper proposes both the member constant and the runtime functions.

§ 6 Using waiting operations in signal handlers

SG1 reviewed P3255R0 and pointed out that, although waiting operations are not lock-free, there exist implementations in which they can be signal-safe because they use a timed backoff approach. David Goldblatt provided an application for calling atomic waiting operations in a signal handler (lightly edited):

> This can be handy with userspace profiling—*e.g.* some "main" thread gets a SIGALRM, and when it gets it, it sends a SIGUSR to whatever set of threads it's interested in, then that SIGUSR handler gathers profiling data and lets the main thread know it's done. You want the main thread to wait until all the signal handlers it triggered have executed.

To support signal handlers that may wish to use atomic waiting operations, this paper also proposes functions and traits to query whether atomic waiting operations are signal-safe,

analogous to the queries of whether atomic notifying operations are lock-free.

§ 7 Proposal

For the foregoing reasons, I propose that:

— the Standard be amended to clarify that the waiting and notifying operations of `std::atomic_flag` are not required to be lock-free, consistent with existing practice;

— the Standard be amended to clarify that `is_lock_free`, `is_always_lock_free`, and `std::atomic_is_lock_free` do not pertain to the atomic waiting and notifying operations, consistent with existing practice; and

— the member function `notify_is_lock_free`, the free function `std::atomic_notify_is_lock_free`, and the member constant `notify_is_always_lock_free` be added for `std::atomic_flag`, `std::atomic`, and `std::atomic_ref`, and that their values indicate whether atomic notifying operations for the corresponding atomic type are lock-free; and

— the member function `wait_is_signal_safe`, the free function `std::atomic_notify_is_signal_safe`, and the member constant `notify_is_always_signal_safe` be added for `std::atomic_flag`, `std::atomic`, and `std::atomic_ref`, and that their values indicate whether atomic waiting operations for the corresponding atomic type may be used in signal handlers without undefined behavior.

I do not propose the addition of macros `ATOMIC_BOOL_NOTIFY_LOCK_FREE` and so on at this time (§33.5.5 [atomics.lockfree]) because the spelling of any such macros would need to be decided by WG14 before they are added to C++ and, in fact, C doesn't even have atomic waiting and notifying operations yet.

§ 8 Wording

In §17.3.2 [version.syn], add a feature test macro named `__cpp_lib_atomic_notify_is_lock_free` with the comment `// freestanding, also in <atomic>`.

Edit §17.13.5 [support.signal]p2:

> A *plain lock-free atomic operation* is an invocation of a function `f` from [atomics] , other than an atomic waiting or notifying operation ([atomics.wait]), such that:
>
> — `f` is the function `atomic_is_lock_free()` , `atomic_notify_is_lock_free()`, or `atomic_wait_is_signal_safe()`, or

— `f` is the member function `is_lock_free()` , `notify_is_lock_free()`, or `wait_is_signal_safe()`, or

— `f` is a non-static member function of class `atomic_flag`, or

— `f` is a non-member function, and the first parameter of `f` has type *cv* `atomic_flag*`, or

— `f` is a non-static member function invoked on an object `A`, such that `A.is_lock_free()` yields `true`, or

— `f` is a non-member function, and for every pointer-to-atomic argument `A` passed to `f`, `atomic_is_lock_free(A)` yields `true`.

Insert a new paragraph after §17.13.5 [support.signal]p2:

A *lock-free atomic notifying operation* is an invocation of an atomic notifying function ([atomics.wait]) `f` such that

— `f` is a non-static member function of a class `A` such that `A.notify_is_lock_free()` yields `true`, or

— `f` is a non-member function whose parameter has type pointer to *cv* `A`, where `A.notify_is_lock_free()` yields `true`.

Edit §17.13.5 [support.signal]p3:

An evaluation is *signal-safe* unless it includes one of the following:

— a call to any standard library function, except for plain lock-free atomic operations, lock-free atomic notifying operations, and functions explicitly identified as signal-safe;
[*Note 1*: […] — *end note*]

— […]

Edit §33.5.2 [atomics.syn]:

```
// ...
template<class T>
  bool atomic_is_lock_free(const volatile atomic<T>*) noexcept;
       // freestanding
template<class T>
    bool  atomic_is_lock_free(const  atomic<T>*)  noexcept;    //
       freestanding
+ template<class T>
+    bool atomic_notify_is_lock_free(const volatile atomic<T>*)
       noexcept;  // freestanding
+ template<class T>
+   bool atomic_notify_is_lock_free(const atomic<T>*) noexcept;
       // freestanding
+ template<class T>
+    bool atomic_wait_is_signal_safe(const volatile atomic<T>*)
       noexcept;  // freestanding
+ template<class T>
```

```
+    bool atomic_wait_is_signal_safe(const atomic<T>*) noexcept;
        // freestanding
// ...
// [atomics.flag], flag type and operations
// struct atomic_flag;  // freestanding

+     bool     atomic_flag_notify_is_lock_free(const     volatile
        atomic_flag*) noexcept;  // freestanding
+    bool    atomic_flag_notify_is_lock_free(const    atomic_flag*)
        noexcept;  // freestanding
+     bool     atomic_flag_wait_is_signal_safe(const     volatile
        atomic_flag*) noexcept;  // freestanding
+    bool    atomic_flag_wait_is_signal_safe(const    atomic_flag*)
        noexcept;  // freestanding

// ...
```

Insert a paragraph before §33.5.5 [atomics.lockfree]p1:

> A *lock-free atomic type* is one for which operations other than atomic waiting and notifying operations ([atomics.wait]) are lock-free.

Edit the synopsis in §33.5.7.1 [atomics.ref.generic.general]:

```
// ...
static  constexpr  bool  is_always_lock_free  =  implementation-
        defined;
bool is_lock_free() const noexcept;
+    static    constexpr    bool    notify_is_always_lock_free    =
        implementation-defined;
+ bool notify_is_lock_free() const noexcept;
+    static    constexpr    bool    wait_is_always_signal_safe    =
        implementation-defined;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit §33.5.7.2 [atomics.ref.ops]p3:

> The static data member `is_always_lock_free` is `true` if ~~the `atomic_ref` type's operations are always~~`atomic_ref<T>` is lock-free ([atomics.lockfree]), and `false` otherwise.

Edit §33.5.7.2 [atomics.ref.ops]p4:

> *Returns*: `true` if ~~operations on all objects of the type `atomic_ref<T>` are~~`atomic_ref<T>` is lock-free ([atomics.lockfree]), `false` otherwise.

Insert four paragraphs after §33.5.7.2 [atomics.ref.ops]p4:

> `static constexpr bool notify_is_always_lock_free;`
> The static data member `notify_is_always_lock_free` is `true` if atomic notifying operations for type `atomic_ref<T>` are lock-free, and `false` otherwise.

`bool notify_is_lock_free() const noexcept;`
Returns: `true` if atomic notifying operations for type `atomic_ref<T>` are lock-free, `false` otherwise.

`static constexpr bool wait_is_always_signal_safe;`
The static data member `wait_is_always_signal_safe` is `true` if atomic waiting operations for type `atomic_ref<T>` are signal-safe ([support.signal]), and `false` otherwise.

`bool wait_is_signal_safe() const noexcept;`
Returns: `true` if atomic waiting operations for type `atomic_ref<T>` are signal-safe, `false` otherwise.

Edit §33.5.7.3 [atomics.ref.int]p1:

```
// ...
static constexpr bool is_always_lock_free = implementation-
        defined;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
        implementation-defined;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
        implementation-defined;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit §33.5.7.4 [atomics.ref.float]p1:

```
// ...
static constexpr bool is_always_lock_free = implementation-
        defined;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
        implementation-defined;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
        implementation-defined;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit the synopsis in §33.5.7.5 [atomics.ref.pointer]:

```
// ...
static constexpr bool is_always_lock_free = implementation-
        defined;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
        implementation-defined;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
        implementation-defined;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit the synopsis in §33.5.8.1 [atomics.types.generic.general]:

```
// ...
static constexpr bool is_always_lock_free = implementation-
      defined;
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
      implementation-defined;
+ bool notify_is_lock_free() const volatile noexcept;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
      implementation-defined;
+ bool wait_is_signal_safe() const volatile noexcept;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit §33.5.8.2 [atomics.types.operations]p4:

> The static data member `is_always_lock_free` is `true` if ~~the `atomic_ref` type's operations are always~~`atomic<T>` is lock-free ([atomics.lockfree]), and `false` otherwise.
> [*Note 2*: […] —*end note*]

Edit §33.5.8.2 [atomics.types.operations]p5:

> *Returns*: `true` if ~~the object's operations are~~`atomic<T>` is lock-free ([atomics.lockfree]), `false` otherwise.
> [*Note 3*: […] —*end note*]

Insert four paragraphs after §33.5.8.2 [atomics.types.operations]p5:

> `static constexpr bool notify_is_always_lock_free = implementation-defined;`
> The static data member `notify_is_always_lock_free` is `true` if atomic notifying operations for type `atomic<T>` are lock-free, and `false` otherwise.
>
> `bool notify_is_lock_free() const volatile noexcept;`
> `bool notify_is_lock_free() const noexcept;`
> *Returns*: `true` if atomic notifying operations for type `atomic<T>` are lock-free, `false` otherwise.
>
> `static constexpr bool wait_is_always_signal_safe = implementation-defined;`
> The static data member `wait_is_always_signal_safe` is `true` if atomic waiting operations for type `atomic<T>` are signal-safe ([support.syn]), and `false` otherwise.
>
> `bool wait_is_signal_safe() const volatile noexcept;`
> `bool wait_is_signal_safe() const noexcept;`

Returns: `true` if atomic waiting operations for type `atomic<T>` are signal-safe, `false` otherwise.

Edit §33.5.8.3 [atomics.types.int]p1:

```
// ...
static constexpr bool is_always_lock_free = implementation-
    defined;
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
      implementation-defined;
+ bool notify_is_lock_free() const volatile noexcept;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
      implementation-defined;
+ bool wait_is_signal_safe() const volatile noexcept;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit §33.5.8.4 [atomics.types.float]p1:

```
// ...
static constexpr bool is_always_lock_free = implementation-
    defined;
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
      implementation-defined;
+ bool notify_is_lock_free() const volatile noexcept;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
      implementation-defined;
+ bool wait_is_signal_safe() const volatile noexcept;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit the synopsis in §33.5.8.5 [atomics.types.pointer]:

```
// ...
static constexpr bool is_always_lock_free = implementation-
    defined;
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
      implementation-defined;
+ bool notify_is_lock_free() const volatile noexcept;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
      implementation-defined;
+ bool wait_is_signal_safe() const volatile noexcept;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit the synopsis in §33.5.8.7.2 [util.smartptr.atomic.shared]:

```
// ...
static constexpr bool is_always_lock_free = implementation-
        defined;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
        implementation-defined;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
        implementation-defined;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit the synopsis in §33.5.8.7.3 [util.smartptr.atomic.weak]:

```
// ...
static constexpr bool is_always_lock_free = implementation-
        defined;
bool is_lock_free() const noexcept;
+   static   constexpr   bool   notify_is_always_lock_free   =
        implementation-defined;
+ bool notify_is_lock_free() const noexcept;
+   static   constexpr   bool   wait_is_always_signal_safe   =
        implementation-defined;
+ bool wait_is_signal_safe() const noexcept;
// ...
```

Edit the synopsis in §33.5.10 [atomics.flag]:

```
namespace std {
struct atomic_flag {
+         static constexpr bool notify_is_always_lock_free =
        implementation-defined;
+     bool notify_is_lock_free() const volatile noexcept;
+     bool notify_is_lock_free() const noexcept;
+         static constexpr bool wait_is_always_signal_safe =
        implementation-defined;
+     bool wait_is_signal_safe() const volatile noexcept;
+     bool wait_is_signal_safe() const noexcept;
// ...
};
}
```

Edit §33.5.10 [atomics.flag]p2:

> ~~Operations on an object of type `atomic_flag` shall be lock-free. The operations should also be address-free.~~ `atomic_flag` is a lock-free type ([atomics.lockfree]).

Drafting note: We shouldn't need to repeat the "should be address-free" recommendation from §33.5.5 [atomics.lockfree]p5.

Insert four paragraphs after §33.5.10 [atomics.flag]p3:

> `static constexpr bool atomic_flag::notify_is_always_lock_free = implementation-defined;`

The static data member `notify_is_always_lock_free` is `true` if atomic notifying operations for type `atomic_flag` are lock-free, and `false` otherwise.

```
bool atomic_flag_notify_is_lock_free(const volatile atomic_flag* object) noexcept;
bool atomic_flag_notify_is_lock_free(const atomic_flag* object) noexcept;
bool atomic_flag::notify_is_lock_free() const volatile noexcept;
bool atomic_flag::notify_is_lock_free() const noexcept;
```

Returns: `true` if atomic notifying operations for type `atomic_flag` are lock-free, `false` otherwise.

```
static constexpr bool atomic_flag::wait_is_always_signal_safe = implementation-defined;
```

The static data member `wait_is_always_signal_safe` is `true` if atomic waiting operations for type `atomic_flag` are signal-safe ([support.syn]), and `false` otherwise.

```
bool atomic_flag_wait_is_signal_safe(const volatile atomic_flag* object) noexcept;
bool atomic_flag_wait_is_signal_safe(const atomic_flag* object) noexcept;
bool atomic_flag::wait_is_signal_safe() const volatile noexcept;
bool atomic_flag::wait_is_signal_safe() const noexcept;
```

Returns: `true` if atomic waiting operations for type `atomic_flag` are signal-safe, `false` otherwise.

§ 9 References

[P0152R1] Olivier Giroux, JF Bastien, Jeff Snyder. 2016-03-02. constexpr atomic<T>::is_always_lock_free.
https://wg21.link/p0152r1

[P0514R0] Olivier Giroux. 2016-11-15. Enhancing std::atomic_flag for waiting.
https://wg21.link/p0514r0

[P0514R1] Olivier Giroux. 2017-06-14. Enhancing std::atomic_flag for waiting.
https://wg21.link/p0514r1

[P1135R6] David Olsen, Olivier Giroux, JF Bastien, Detlef Vollmann, Bryce Lelbach. 2019-07-20. The C++20 Synchronization Library.
https://wg21.link/p1135r6

[P1643R1] David Olsen. 2019-07-20. Add wait/notify to atomic_ref.
https://wg21.link/p1643r1

[refimpl] NVIDIA Corporation. 2016-11-11. ogiroux/atomic_flag.
https://github.com/ogiroux/atomic_flag/blob/master/atomic_flag/atomic_flag.hpp

1. Although `notify_one` is supposed to wake up only one waiting thread, the specification of `wait` allows for spurious wakeups. Therefore, an implementation of `wait` that spins and returns whenever it sees that the value has changed is conforming: if `notify_one` was called, then one of the threads that wakes up can be arbitrarily considered to have been notified, while all other such threads can be considered to have been unblocked spuriously. ↩

2. All citations to the Standard are to working draft N4981 unless otherwise specified. ↩